Permissions Architecture

Permission Levels: Architecture and Implementation Plan

Project: Cashflow application, ADT Dental Clinic **Date:** November 3, 2026 **Status:** Proposal for review. Nothing in the app has been changed yet. **Decisions already made:** fixed permission table in the code (not editable in the app); staff are limited to their own branch everywhere; this document only (no build yet).

---

1. Summary

The app already has three roles (**admin**, **dentist**, **staff**) and most sensitive actions are admin-only. But the rules are written as about 30 separate `role === "admin"` checks spread over pages, server actions and API routes, so there is no single place that says who may do what, and it is easy to forget a check when something new is added. A review of the code also found gaps that matter more once staff are limited to their own branch:

#	Gap found	Risk
G1	Any signed-in user can **change their own branch** from My profile (`updateMyBranchAction`).	A staff member can move themselves to any branch, which would defeat branch limits.
G2	Any signed-in user can **import a CSV** (`importCsvAction`).	Bulk creation or overwriting of visits by a front-desk account.
G3	Lists and visit pages do not check the **branch**: Transactions, Patients, Queue, Expenses, a visit's page, attachments (`/api/cashflow/attachments/[id]`).	A staff member of one branch can read another branch's figures, or open its receipts by ID.
G4	Visit actions (edit, status, payment, attach, remove from queue) take an ID and do not check that the visit belongs to the user's branch.	Same problem when an action is called directly, not only through the screen.
G5	`updateInvoiceAction` is not role-limited (only "locked once Cleared" is enforced); the Edit page is admin-only but the action behind it is not.	A direct call can edit an open visit that the screen would not let staff edit.
G6	Staff with **no branch** assigned fall back to the first branch on Day end, and see everything elsewhere.	The limit silently does not apply.
G7	The role is read from the sign-in token and only refreshed on a "Stay signed in" refresh.	A demoted user keeps admin rights until the sign-in token is next refreshed.
G8	The sign-in proxy checks sign-in only, not role; admin pages protect themselves one by one.	Fine today, but one forgotten page check exposes an admin screen.

Proposal in one paragraph. Put the rules in **one table** (`permissions.ts`: permission × role) with one function, `can(role, permission)`. Every page, action and API route calls one **guard** (`requirePermission`) at the top. A second small piece, the **branch scope**, decides which rows a user may see and is applied inside the database queries, so forgetting it is not possible. The menu, buttons and the "Who can do what"

screen are all drawn from the same table. A test fails the build if an action or route has no guard. Rollout is in five phases; the first closes G1 and G2 in half a day.

---

2. How it works today (as built)

Roles (`users.role`): `admin`, `dentist`, `staff`. The role is put in the sign-in token and in the session (`session.user.role`). The branch is read from the `users` table (`branchId`, may be empty).

Where rules are enforced today

1. **Pages** check the role and redirect (Users, Branches, Commissions, Commission report, Edit transaction, Audit tab).
2. **Server actions** check the role and return a message ("Only admins can ...").
3. **API routes** return 401 / 403 (commission report download, attachment delete).
4. **Screens** hide buttons and menu items (menu lists differ for admin, dentist and staff).
5. **Queries**: only Day end (`resolveDayEndScope`) and the Overview limit staff to their own branch.

What each role can do now

Area	Admin	Dentist	Staff
Overview	All branches, edit sales targets	Own branch (no targets)	Own branch (no targets)
Queue / add visit / check in	Yes	Yes	Yes
Move a visit status	Any move, with a reason	One step forward	One step forward
Remove a visit from the Queue	Any visit	Arrived / On-going only	Arrived / On-going only
Delete a visit or transaction	Yes	No	No
Edit a visit	Edit page and pop-up	Open visits, by direct action (G5)	Open visits, by direct action (G5)
Import CSV	Yes	Yes (G2)	Yes (G2)
Add a payment	Yes	Yes	Yes
Delete a payment	Yes	No	No
Patients: add, edit	Yes	Yes	Yes
Patients: merge	Yes	No	No
PWD / Senior ID numbers	Full	Last 4 only	Last 4 only
Expenses: add	Yes	Yes	Yes
Expenses: edit, delete	Yes	No	No
Attachments: view, upload	Yes	Yes (any branch, G3)	Yes (any branch, G3)
Attachments: delete	Yes	No	No
Price list (Procedures tab)	Edit	Read	Read
Audit trail (tab and per visit / patient)	Yes	No	No
Day end and download	Any branch	Own branch	Own branch
Commission rates, tiers, report	Yes	No	No

Area	Admin	Dentist	Staff
Performance	Everything	Own numbers	Own bonus
Users, Branches	Yes	No	No
My profile: theme / own branch	Yes / yes	Yes / yes (G1)	Yes / yes (G1)

Dentist and staff differ only in what Performance shows them and in how commission is linked to a dentist. For permissions they are the same level, so this design treats them as one **non-admin** level with a dentist-only Performance view.

---

3. Design principles

1. **One table is the truth.** Rules live in a single file that a person can read in a minute and review with the clinic.
2. **Deny by default.** A permission that is not in the table is refused; an unknown role has nothing.
3. **The server decides.** Hiding a button is a courtesy; every action and route re-checks. The browser never sends a role or a branch that is trusted.
4. **Scope inside the query.** Branch limits are part of the database query (a `where`), not a filter applied to a list afterwards, and a visit ID from another branch behaves as "not found".
5. **Same table everywhere.** Menu, buttons, guards, the matrix screen and the tests all read the same table, so they cannot drift apart.
6. **Fresh on every request.** The role and branch come from the database once per request, so a demotion or a branch change applies at once.

---

4. The model

Three small ideas:

- **Role**: what a person is (admin, dentist, staff). Stored on the user.
- **Permission**: one named capability, such as `expense.delete`. Roles are granted permissions in the table.
- **Scope**: which branches the permission covers. Admins: **all branches**. Everyone else: **their own branch only**; a non-admin with no branch assigned has **no scope** (sees nothing and is asked to contact an admin). *Self* scope (own numbers) is used only for Performance.

4.1 Permission table (proposed)

"All" means every signed-in role. **Changed** marks a difference from today.

Permission	What it allows	Admin	Dentist	Staff	vs today
`overview.view`	Overview (scope limited)	Yes	Yes	Yes	same
`targets.edit`	Edit sales targets	Yes			same
`queue.view`	Queue board	Yes	Yes	Yes	same
`visit.create`	Add a visit / check in	Yes	Yes	Yes	same (branch forced to own)
`visit.edit_open`	Edit a visit that is not Cleared	Yes	Yes	Yes	decide D2

Permission	What it allows	Admin	Dentist	Staff	vs today
`visit.edit_any`	Edit any visit, incl. Cleared, Edit page	Yes			**changed** (enforced in the action)
`visit.status.advance`	Move one step forward	Yes	Yes	Yes	same
`visit.status.override`	Move back or skip, with a reason	Yes			same
`visit.remove_from_queue`	Remove Arrived / On-going	Yes	Yes	Yes	same (state rules unchanged)
`visit.delete`	Delete a visit or transaction	Yes			same
`transactions.import_csv`	Import a CSV	Yes			**changed** (G2)
`payment.add`	Record a payment	Yes	Yes	Yes	same
`payment.delete`	Remove a payment	Yes			same
`attachment.view`	Open a receipt / file (scope limited)	Yes	Yes	Yes	**changed** (G3)
`attachment.upload`	Attach a file	Yes	Yes	Yes	same (scope limited)
`attachment.delete`	Delete a file	Yes			same
`patient.view`	Patient list and profile	Yes	Yes	Yes	see D1
`patient.create` / `patient.edit`	Add and correct a patient	Yes	Yes	Yes	same
`patient.merge`	Merge duplicates	Yes			same
`patient.ids.reveal`	See PWD / Senior ID in full	Yes			same
`expense.view`	Expenses list (scope limited)	Yes	Yes	Yes	**changed** (G3), see D3
`expense.add`	Add an expense	Yes	Yes	Yes	same (branch forced to own)
`expense.edit` / `expense.delete`	Correct or delete an expense	Yes			same
`dayend.view` / `dayend.download`	Day end report and files	Yes	Yes	Yes	same (scope limited)
`dayend.all_branches`	Pick another branch	Yes			same
`performance.view_own`	Own performance / bonus		Yes	Yes	same
`performance.view_all`	Everyone's performance	Yes			same
`procedure.view`	Price list, read only	Yes	Yes	Yes	same
`procedure.manage`	Edit prices, add / hide procedures	Yes			same
`commission.manage`	Rates, tiers, dentist tiers	Yes			same
`commission.report`	Commission report and downloads	Yes			same
`audit.view`	Audit trail	Yes			same
`user.manage`	Create, change role / branch, delete users	Yes			same
`branch.manage`	Create, rename, delete branches	Yes			same

Permission	What it allows	Admin	Dentist	Staff	vs today
`profile.edit_self`	Theme and name	Yes	Yes	Yes	same
`profile.change_branch`	Change **own** branch	Yes			**changed** (G1)

Rules that stay in the code next to the permission, because they depend on the visit and not only on the role: the status ladder (`canMove`), the Cleared lock, "balance must be paid", "LOA before Cleared", and the Queue removal states (`canRemoveFromQueue`). The permission says *may this role try*; those rules say *is it allowed for this visit now*.

4.2 Branch scope by area

Area	Admin	Dentist and staff	Edge cases
Transactions list, visit page, invoice, edit	All	Own branch; another branch's visit ID is "not found"	Visits with no branch (old data): admin only
Queue board	All (can filter)	Own branch	
Add a visit	Any branch	Branch is set to the user's own, any posted value is ignored	
Payments, status, remove, attachments	Any	Only on own-branch visits	
Patients	All	Patient records are shared (see D1); a patient's **visits and amounts** are limited to own branch	New patient with no visit yet is visible to all
Expenses	All	Own branch only	Clinic-wide expenses (no branch): admin only (D3); staff add always to own branch
Day end	Any branch	Own branch (already built)	No branch assigned: no data, not "first branch" (G6)
Overview	All	Own branch (already built)	
Performance	All	Self only	
Commission, users, branches, audit	Admin only	None	

---

5. Code architecture

```
Browser (menu, buttons)      reads  can()  (hidden, never trusted)
        │
Proxy (proxy.ts)             signed in? optional role check on admin-only prefixes  (UX + defens
e in depth)
        │
Page / Server action / API route
        │   requirePermission("expense.delete")  ->  actor { id, role, branchId } or a denial
        │   assertVisitInScope(actor, invoiceId) ->  not found if another branch
        ▼
Data layer                   every query takes a Scope  ->  where branch_id = scope.branchId
```

`        |`
`Database`

5.1 New files

File	Purpose
`src/lib/auth/permissions.ts`	The table (`PERMISSIONS`), `Permission` type, `can(role, permission)`, `permissionsFor(role)`. Pure, no database.
`src/lib/auth/actor.ts`	`getActor()`: the signed-in user's `{ id, name, role, branchId }`, loaded **once per request** (React `cache`) from the database. Replaces the repeated `getUserById` calls.
`src/lib/auth/guards.ts`	`requirePermission(p)` for server actions (returns the actor or a denial), `requirePagePermission(p)` (redirects), `requireRoutePermission(p)` (401 / 403 JSON).
`src/lib/auth/scope.ts`	`scopeFor(actor)` giving `{ kind: "all" } | { kind: "branch", branchId } | { kind: "none" }`; `branchWhere(scope, column)`; `assertVisitInScope`, `assertExpenseInScope`.
`src/components/auth/can.tsx`	`<Can permission="expense.edit">` for screens; the server passes the permission list, the client never computes a role itself.
`scripts/check-permissions.mjs`	Build-time check: every `"use server"` export and every `route.ts` handler must call a guard (or be on a short, reviewed allow-list such as the health check and sign-in).
`src/components/cashflow/permission-matrix.tsx`	Read-only "Who can do what" screen on the Users page, drawn from the table.

5.2 The table

```ts
// src/lib/auth/permissions.ts
import type { UserRole } from "@/lib/cashflow/user-schema";

const ALL = ["admin", "dentist", "staff"] as const;
const ADMIN = ["admin"] as const;
const NON_ADMIN = ["dentist", "staff"] as const;

export const PERMISSIONS = {
  "expense.view": ALL,
  "expense.add": ALL,
  "expense.edit": ADMIN,
  "expense.delete": ADMIN,
  "transactions.import_csv": ADMIN,
  "profile.change_branch": ADMIN,
  "performance.view_own": NON_ADMIN,
  // ... every row of section 4.1
} as const satisfies Record<string, readonly UserRole[]>;

export type Permission = keyof typeof PERMISSIONS;

export function can(role: UserRole | undefined, permission: Permission): boolean {
  return role !== undefined && (PERMISSIONS[permission] as readonly UserRole[]).includes
```

```
(role);
        }
```

5.3 Using it

Server action (replaces `if (session.user.role !== "admin")`):

```
        export async function deleteExpenseAction(id: string) {
          const gate = await requirePermission("expense.delete");
          if (!gate.ok) return gate;                        // { ok:false, message }
          await assertExpenseInScope(gate.actor, id);       // branch check (admin passes)
          ...
        }
```

Page: `const actor = await requirePagePermission("commission.report");` (redirects to the dashboard when refused).

API route: `const gate = await requireRoutePermission("attachment.view"); if (!gate.ok) return gate.response;`

Queries: functions that read visits, expenses, queue or attachments take a scope and apply it in the `where`:

```
        listTransactionsPage({ ...filters, scope })          // where branchWhere(scope, invoices.branchId)
        getInvoiceForActor(actor, invoiceId)                 // undefined (page shows "not found") outside scope
```

For `kind: "none"` the helper produces a condition that matches nothing and the page shows "Ask an admin to assign your branch."

Screen: the layout loads the actor and passes `permissions` (a list of strings) to the menu and to client components; the menu entries carry a permission instead of three hard-coded arrays.

Writes force the branch: `createInvoiceAction` and `addExpenseAction` set `branchId = actor.branchId` for non-admins and ignore the posted value.

5.4 Role and branch always fresh (G7)

`getActor()` reads the user row once per request (`cache()` so pages, layout and actions share it). A demoted or moved user is affected on their next click, a deleted user is signed out (the existing 30-minute token rule stays as the outer limit). This also removes the repeated user lookups that showed up in the page-speed review (Transactions reads the user again after the layout did).

5.5 Proxy as a second layer (G8)

`proxy.ts` already knows whether the person is signed in. Add a list of admin-only path prefixes (`/cashflow/users`, `/cashflow/branches`, `/cashflow/commissions`) taken from the same table and redirect non-admins there. Next's own guidance is that the proxy is not a full authorization layer, so this is an early, cheap redirect; the real checks stay in the page, action and route.

5.6 Audit of refusals

A refused permission check writes one line to the audit trail (`access.denied`, who, what permission, which page or action). Repeated refusals by one user are then visible to the admin and show who is probing. Never logged: the contents of the request.

---

6. Database changes

None are needed for the permission table (it is code). Two small, optional ones:

1. **A branch for every non-admin.** Today a branch may be empty. Add a check in *Create user* and *Change role* that a dentist or staff user must have a branch, and a data check before go-live:

   ```
   select name, email, role from users where role <> 'admin' and branch_id is null;
   ```

   Any row returned needs a branch set on the Users page, or that person will see no data once scope is enforced.

2. `users.active` (separate item in the review, "disable user") fits naturally here: `getActor()` would refuse an inactive user. Not required for this work.

---

7. Rollout plan

Estimates are rough working days for one developer including checks.

Phase	What	Why this order	Files	Est.
0. Close the open gaps	Admin-only own-branch change (G1); admin-only CSV import (G2); `updateInvoiceAction` enforces `visit.edit_any` for Cleared and the Edit page flow (G5)	Smallest changes with the most risk removed; no new structure needed	`profile/actions.ts`, `actions.ts`, profile page	0.5
1. Permission module	`permissions.ts`, `actor.ts`, `guards.ts`; move the ~30 existing checks onto `requirePermission`; menu and tabs read the table; no change in behavior except phase 0	One place for the rules before adding more	all files in Appendix A	1.5
2. Branch scope	`scope.ts`; scope parameter in the visit, queue, expense, patient-history and attachment queries; `assertVisitInScope` in every visit action; force branch on writes; "no branch" message; branch required for non-admins	Fixes G3, G4, G6; the largest change, done once the guards exist	`db/transactions.ts`, `db/invoices.ts`, `db/expenses.ts`, `db/attachments.ts`, `db/patient-admin.ts`, queue, patient and visit pages, attachment routes	2
3. Screens and visibility	`<Can>`, permission list from the layout, "Who can do what" matrix on the Users page, branch label on scoped pages	Makes the rules visible to the clinic	components	0.5
4. Defense in depth and tests	Proxy prefix check, `access.denied` audit lines, `check-permissions.mjs` in the build, tests below	Locks the result in against future changes	`proxy.ts`, `audit.ts`, scripts	1

Total about **5.5 days**. Phases 0 and 1 can ship alone and already remove most of the risk; phase 2 should ship together with the data check in section 6 and a note to the branches.

Going live safely. Run the section 6 query first. Ship phase 2 in the evening; keep an admin signed in on a second device. If a branch sees nothing, the cause is a missing branch on the user, fixed on the Users page in seconds.

---

8. Tests

Automatic

1. **The table itself**: for every permission and role, the expected yes / no (a snapshot of section 4.1), so changing a rule is a visible one-line diff in review.
2. **Coverage check** (`check-permissions.mjs`): fails if a server action or route handler has no guard and is not on the reviewed allow-list.
3. **Scope helper**: `all`, `branch`, `none`; the generated condition for each; a branch ID that does not exist.
4. **Existing rule tests stay** (status ladder, Cleared lock, queue removal states).

By hand, with three accounts (an admin, a staff user at Marikina, a staff user at Antipolo, plus a dentist at Marikina)

Check	Expected
Staff at Marikina opens Transactions, Queue, Patients, Expenses, Day end, Overview	Only Marikina visits, expenses and figures
Staff pastes the URL of an Antipolo visit, its invoice, its edit page	"Not found"
Staff opens an Antipolo receipt link (`/api/cashflow/attachments/<id>`)	Refused / not found
Staff adds a visit with a posted `branchId` of Antipolo (browser tools)	Saved under Marikina
Staff changes own branch on My profile	Not possible (control hidden, action refused)
Staff imports a CSV	Not possible
Staff tries Users, Branches, Commissions, commission report, audit tab by URL	Redirected; the commission download returns 403
Staff edits or deletes an expense, deletes a payment, merges patients	Refused
Admin demotes the staff user while signed in; staff clicks any page	Treated as staff at the next click (G7)
Staff with no branch	A clear "ask an admin to assign your branch" page, no data
Admin opens everything	Unchanged from today
Dentist opens Performance	Own numbers only; no other section

---

9. Decisions to confirm

#	Question	Proposed default
D1	**Patients are shared across branches** (one record, no branch). Should staff still search and open any patient?	Yes: one record avoids duplicate patients and lets a branch check in someone who normally goes elsewhere. Staff see the

#	Question	Proposed default
		patient's details but only that patient's visits and amounts at their own branch. Say if patient details should also be hidden.
D2	May staff and dentists **edit an open visit** (not yet Cleared), for example to add a procedure during treatment? The screen hides the Edit page from them, but the action allows it today.	Yes for open visits (`visit.edit_open`), admin only once Cleared.
D3	**Clinic-wide expenses** (no branch, such as a shared software subscription): can staff see them?	Admin only; staff see their own branch's expenses.
D4	A staff or dentist with **no branch assigned**.	No data and a message; branch required when creating the user.
D5	Should **refused attempts** be written to the audit trail (section 5.6)?	Yes.
D6	Should the dentist keep the same level as staff, or a separate set of permissions?	Same level; they differ only in the Performance view. A separate set can be added to the table later without changing the code.

---

10. Not in this plan, and how it could grow

- **Admin-editable permissions in the app** (an admin switches rules on and off) was considered and not chosen. If it is wanted later, the table moves to a `role_permissions` database table and `can()` reads it (cached, refreshed on change); callers do not change. It would need its own admin-only screen with every change audited and a safeguard so the last admin cannot remove their own access.
- **More roles** (owner, front desk, cashier, branch manager): add the role to the user enum and a column to the table. The branch-scope helper already supports a "branch manager" shape (own branch, more permissions).
- **Per-user exceptions** and **two-step sign-in for admins** are separate items from the October review.

Appendix A. Existing checks to move onto the guards (as of November 3)

Server actions: `patients/actions.ts` (merge), `commissions/actions.ts` (all rates and tiers), `expense-actions.ts` (edit, delete), `transactions/procedure-actions.ts`, `transactions/status-actions.ts` (override), `transactions/payment-actions.ts` (delete), `actions.ts` (remove transaction, remove visit, edit, import, create), `targets-actions.ts`, `users/actions.ts` (all four), `branches/actions.ts` (all three), `profile/actions.ts` (own branch), `patient-actions.ts` (search and lookup: scope only).

Pages: `cashflow/page.tsx` (Overview scope, targets), `patients/[id]/page.tsx` (audit, ID reveal), `day-end/page.tsx`, `commissions/page.tsx`, `commissions/report/page.tsx`, `transactions/page.tsx` (audit tab, edit, delete, Expenses), `transactions/[id]/page.tsx`, `transactions/[id]/edit/page.tsx`, `performance/page.tsx`, `queue/page.tsx`, `users/page.tsx`, `branches/page.tsx`.

API routes: `commissions/report`, `attachments` (upload), `attachments/[id]` (view, delete), `day-end`, `open-visits`.

Screens and logic with role in them: `cashflow-nav.tsx` (three arrays), `transactions-tabs.tsx` (admin-only tabs), `cashflow/layout.tsx`, `today-board.tsx`, `visit-status-control.tsx` and `lib/cashflow/visit-status.ts` (`isAdmin` in `canMove`), `lib/db/day-end.ts` (`resolveDayEndScope`), `lib/db/invoices.ts` (Cleared lock), `auth.ts` (role in the token).

Addendum (Nov 5): the superuser role

The clinic has exactly **one superuser**, who holds every permission and can manage admins. In the permission table (`permissions.ts`) this is a fourth role, `superuser`, listed with all permissions, so the matrix becomes superuser > admin > dentist > staff. Only one account may hold it (database unique index), and the app never creates or removes it. Admins keep their current access but cannot manage other admins or the superuser; this is spelled out for password resets in `PASSWORD_CHANGE_RESET_ARCHITECTURE.md`, section 5A. Decision D7 below should be read with that rule.